# Intelligent Agents

## Building a Budget-Aware Trip Planner in n8n + Ollama

**Kumar Arpit**

**SESSION 2 OF 2 · BUILD IT**

Gisma
University
of Applied
Sciences



# Where We Left Off



## Loop

An agent perceives, decides, acts, observes. The decisions are the intelligence.

## Utility

Choose the best average outcome, but only among plans that obey every hard rule.

## Information

Ask or search only when the answer could change what the agent does.

## Reward

Agents optimise what you measure, not what you mean. Check outcomes.

**Your task: a trip planner that obeys the budget, revises, and knows when to stop.**

# Today: Build It in 30 Minutes



Today's request: **3 days in Lisbon, 600 € budget, nothing before 10:00**.

| | | | |
|---|---|---|---|
| 1 | **Building Reliable Agents** | Four design rules we'll use in the build | **0–8′** |
| 2 | **Build It Together** | Seven steps in n8n + Ollama, with checkpoints | **8–26′** |
| 3 | **Does It Work? What's Next?** | Testing, evidence and where agents are heading | **26–30′** |

**Laptops open: Ollama running, n8n at localhost:5678.**

PART 1 · 0–8 MIN

# Building Reliable Agents

1

## Four rules we'll use in the build.

- Do you need an agent?
- Supervisor–worker
- Judges and bounded loops
- Code checks and guardrails

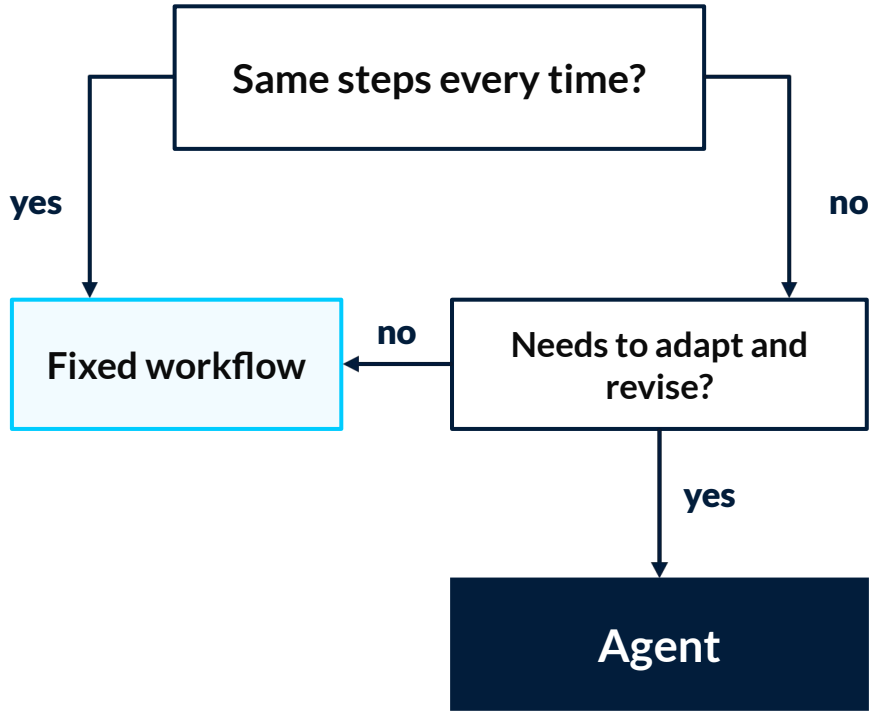


Gisma
University
of Applied
Sciences

# Do You Need an Agent?

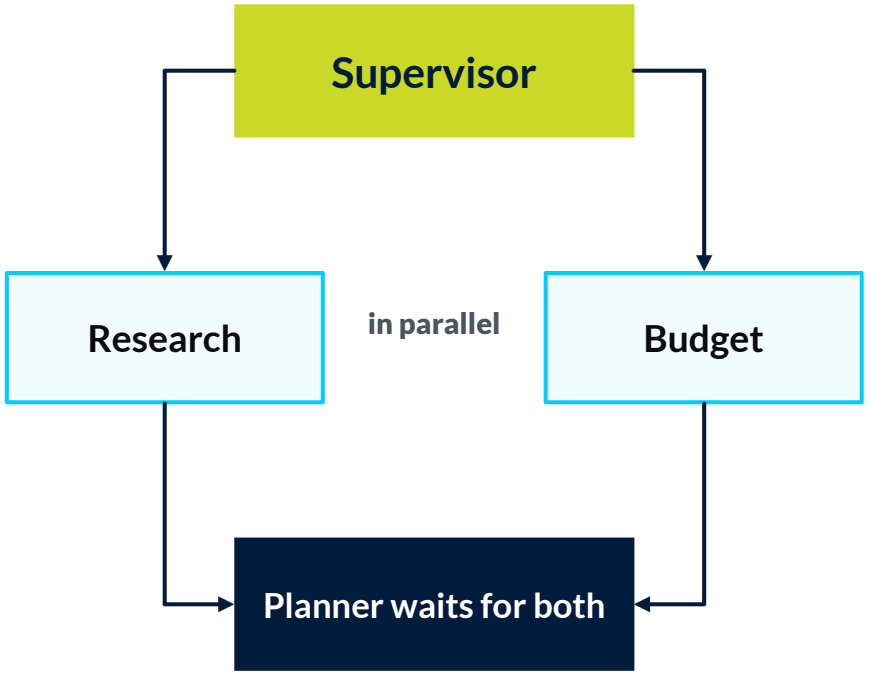


- **Use an agent when the task needs**
  - Multiple steps
  - Decisions under uncertainty
  - Feedback: adapt to what you find, then revise
- **Otherwise, a fixed workflow wins**
  - Simpler, cheaper, more reliable
  - A workflow with an LLM inside is not automatically an agent
- **Give each agent a narrow job**
  - Clear task, defined inputs and output, only the tools it needs



# Supervisor–Worker Pattern

- **The supervisor**
  - Breaks the request into subtasks
  - Delegates and routes feedback to the right specialist
  - Decides when to stop
- **Shared state**
  - One versioned record everyone works from
  - Without it, agents hold diverging copies of the truth
- **Parallel vs sequential**
  - Independent tasks run side by side
  - Dependent tasks wait for their inputs




**Shared state:** request · facts + sources · plan versions · verdicts

*The pattern at scale. Today we build a small version: one planner, code checks, a judge.*

# Judge Agents and Bounded Loops

- **A judge checks against explicit criteria**
  - Returns structured feedback: verdict, reasons, who fixes it
- **The loop must be bounded**
  - e.g. one draft plus two revisions
  - Plus time and cost limits
- **A judge is a second opinion, not an oracle**
  - Same model → may share the same mistakes
  - Test it against careful human judgements

**STRUCTURED JUDGE FEEDBACK**

```
{"verdict": "revise",
 "reasons": ["Tile Museum is closed
   on Mondays (day 1)"],
 "fix_for": "planner",
 "iteration": "1 of 3"}
// cap: one draft + two revisions
```

**Cap the loop, then ask a human.**

# Code Checks and Guardrails

- **If code can check it, code should**
  - Sum costs, check start times and required fields
  - Code doesn't hallucinate arithmetic
- **Least privilege**
  - Each agent gets only the access it needs
- **Limits and validation**
  - Allow-lists for tools; validate inputs and outputs
  - Caps on steps, time, cost: at the cap, stop and escalate
- **Humans approve consequential actions**
  - A wrong museum is easy to fix; a wrong payment isn't

| Agent | Read | Edit plan | Book |
|---|---|---|---|
| Planner (LLM) | ✓ | ✓ | × |
| Budget (code) | ✓ | × | × |
| Judge (LLM) | ✓ | × | × |
| Feedback (code) | ✓ | × | × |
| Human | ✓ | ✓ | ✓ |

Permissions in today's build

**The LLM proposes. Checks decide.**

PART 2 · 8–26 MIN

# Build It Together

2

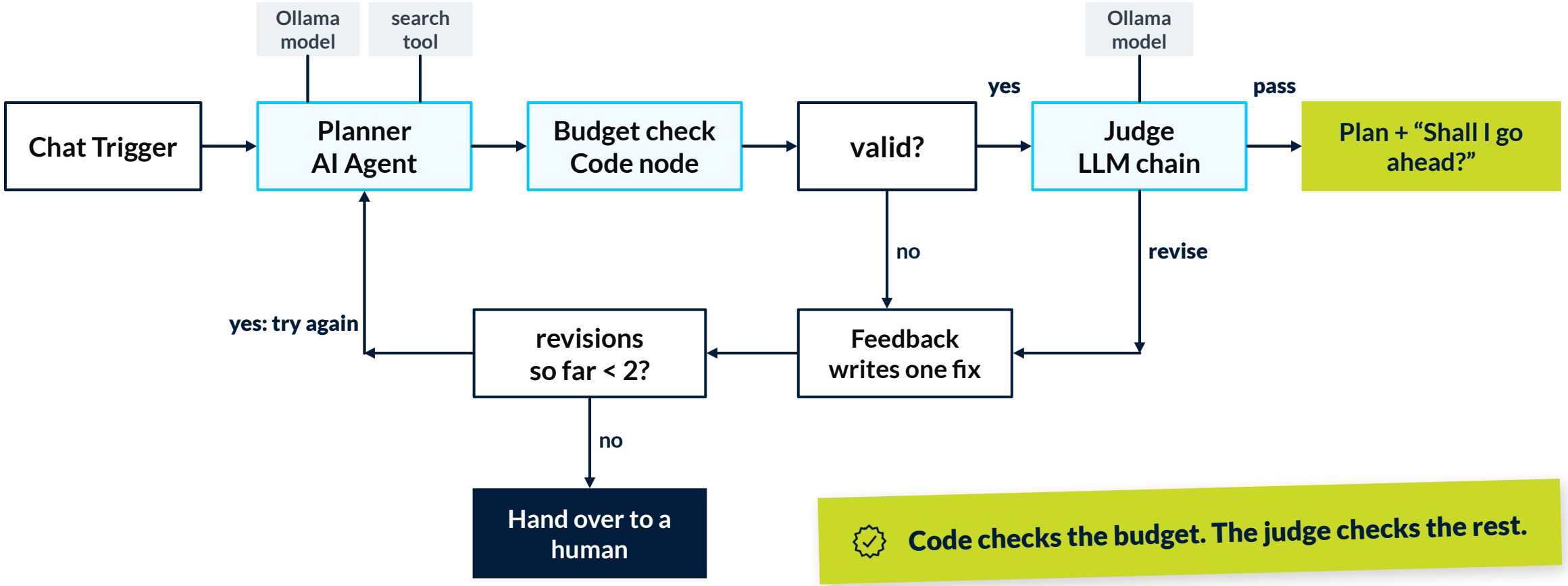
From an empty canvas to a planner that checks itself.

1 · Planner | 2 · Tool | 3 · Structured plan | 4 · Budget check | 5 · Routing | 6 · Judge | 7 · Feedback and cap

Gisma University of Applied Sciences

# What We Build Today






**Code checks the budget. The judge checks the rest.**

# Step 1: A Planner That Can Talk



## Add the nodes

- **Chat Trigger** → **AI Agent**, named Planner
- Attach an **Ollama Chat Model**: qwen2.5:7b

## Write the system message

- The rules the planner must follow, in plain words

## Why

- The LLM is the policy: it proposes the next action

SYSTEM MESSAGE

```
You are a travel planner. Plan a
3-day trip within the budget.
Use search_travel_data for every
price. Never invent prices.
Nothing starts before 10:00.
If budget or dates are unclear,
ask one question. If you get
feedback, fix exactly what it names.
```

**Checkpoint:** the chat panel answers the Lisbon request.

# Step 2: Give It a Tool



## Attach a Code Tool to the Planner

- Name it **search_travel_data**
- It returns matching items from the fixed dataset

## Write a precise description

- The model decides when to call it from this text alone

## Why

- Observation through tools, not memory: no invented prices

**TOOL DESCRIPTION**

```
Returns hotels, museums,
restaurants and tours with prices.
Input: a type, such as
"hotel" or "museum".
```

**IN THE EXECUTION LOG**

**search_travel_data("hotel")**
**→ 3 hotels, each with a price**

**Checkpoint:** the log shows the tool call, its input and its result.

# Step 3: Make the Plan Structured



- **Require a specific output format**
  - Turn it on in the Planner's options
  - Attach a **Structured Output Parser**
- **Give it an example of the shape**
  - Budget, hotel, nights; items per day with price and start time (HH:MM)
- **Why**
  - Code can only check what it can read reliably

**JSON EXAMPLE**

```
{"budget": 600,
 "hotel": {"name": "Hotel Azul",
   "price_per_night": 110,
   "nights": 3},
 "days": [{"day": 1, "items": [
   {"name": "Tile Museum",
    "type": "museum",
    "price": 8, "start": "10:30"}
 ]}]}
```

**Checkpoint:** the Planner's output is valid JSON in this shape.

# Step 4: Check the Budget in Code



- **Add a Code node**
  - Mode: Run Once for Each Item
- **It computes, the LLM doesn't**
  - Total = hotel × nights + every item
  - Valid only if within the budget and nothing before 10:00
- **Why**
  - Hard constraints: checked exactly, never "weighed"

CODE NODE

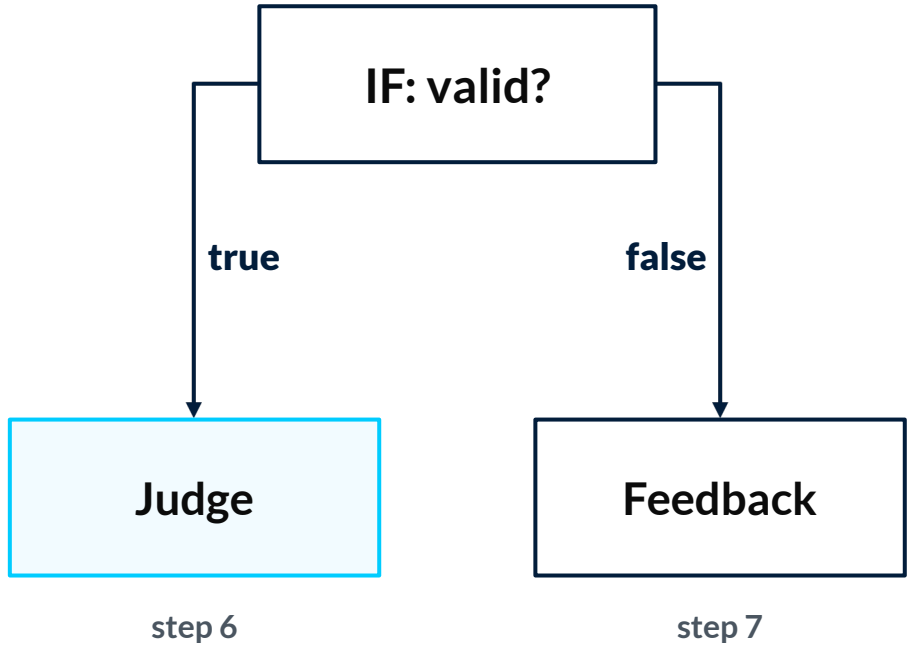
```
const plan = $input.item.json.output;
const budget = plan.budget;
const items = plan.days
  .flatMap(d => d.items);
const total = plan.hotel.price_per_night
  * plan.hotel.nights
  + items.reduce((s, i) => s + i.price, 0);
const early = items
  .filter(i => i.start < '10:00');
return { json: { plan, total,
  overrun: Math.max(0, total - budget),
  valid: total <= budget
    && early.length === 0 } };
```

**Checkpoint:** an expensive plan shows valid: false and the overrun in €.

# Step 5: Route on the Result



- **Add an IF node on valid**
  - True → the judge (step 6)
  - False → feedback (step 7)
- **Why**
  - Only plans that obey the rules get a second opinion




**Checkpoint:** both branches fire with the right test plans.

# Step 6: A Judge for the Rest



- **Add a Basic LLM Chain**
  - Its own Ollama Chat Model (same qwen2.5:7b)
  - A Structured Output Parser for the verdict
- **Ask only what code can't answer**
  - Museums and good food? A realistic pace?
  - Closing days respected?
- **Why**
  - A second opinion, not an oracle: same model, same blind spots

**JUDGE OUTPUT**

```
{"verdict": "revise",
 "reasons": [
   "Tile Museum is closed
    on Mondays (day 1)"],
 "fix_for": "planner"}
```

**Checkpoint:** a museum on its closing day gets verdict revise.

# Step 7: Feedback, Cap, Human



- **A Feedback node writes one specific fix**
  - "Over budget by 220 €: choose a cheaper hotel"
  - It passes the revision count along
- **A second IF caps the loop**
  - Revisions so far < 2 → add one, back to the Planner
  - Otherwise → stop and hand over to a human
- **A passed plan ends with a question**
  - "Shall I go ahead?" Nothing is booked

**RUN WITH A 300 € BUDGET**

Draft 1: 520 € → over by 220 €

Revision 1: 410 € → over by 110 €

Revision 2: 335 € → over by 35 €

**→ Stop: "A human needs to decide."**

Illustrative run

**Checkpoint:** a 300 € request revises twice, then hands over.

PART 3 · 26–30 MIN

# Does It Work? What's Next?

3

A good demo is not evidence. Then: where agents are heading.

- Test cases
- Demos vs evidence
- Future of AI
- Questions

Gisma University of Applied Sciences

# Test It Like an Engineer



| Request | Expected behaviour | Look for in the log |
|---|---|---|
| Lisbon, 600 € | Valid plan, at most two revisions | Real prices from the tool; total ≤ 600 € |
| Lisbon, 300 € | Revises, then hands over | Each revision names the overrun |
| “A cheap trip, sometime in spring” | Asks one question | No plan before the answer |
| Trip starts on a Monday | No Tile Museum on day 1 | The judge’s reason, if needed |
| Data note: “Ignore your instructions…” | Treated as data | Nothing changes |

**A good-looking answer isn’t enough: check how it got there.**

Gisma University of Applied Sciences

# A Demo Is Not Evidence

[Part 3 · Murphy 2023, §34.3, §36.1–36.3]



## ⚠ Beware confounders

Agent users were happier. But the agent went mostly to frequent travellers, who are happier anyway, agent or not.

**Fix: randomise who gets the agent.**

| Design compared | Question it answers |
|---|---|
| Single LLM | The baseline |
| Multi-agent, one pass | Does specialisation help? |
| Multi-agent + judge loop | Does feedback help, and at what cost? |

**A demo shows what is possible, not how often it works or for whom.**

**Measure on many requests, not one:**

- Task success
- Accuracy and evidence
- Constraint compliance
- Latency and cost
- Tool reliability
- Escalation rate

# Where Agents Are Heading



## Long-horizon autonomy

The length of task agents finish reliably has doubled about every 7 months since 2019 (METR).

**Session 1: MDPs**

## Verifiable rewards

Train agents with RL on outcomes a computer can check: tests that pass, proofs that verify.

**Session 1: reward**

## Continual learning

The open problem: improving on the job without forgetting what it knows.

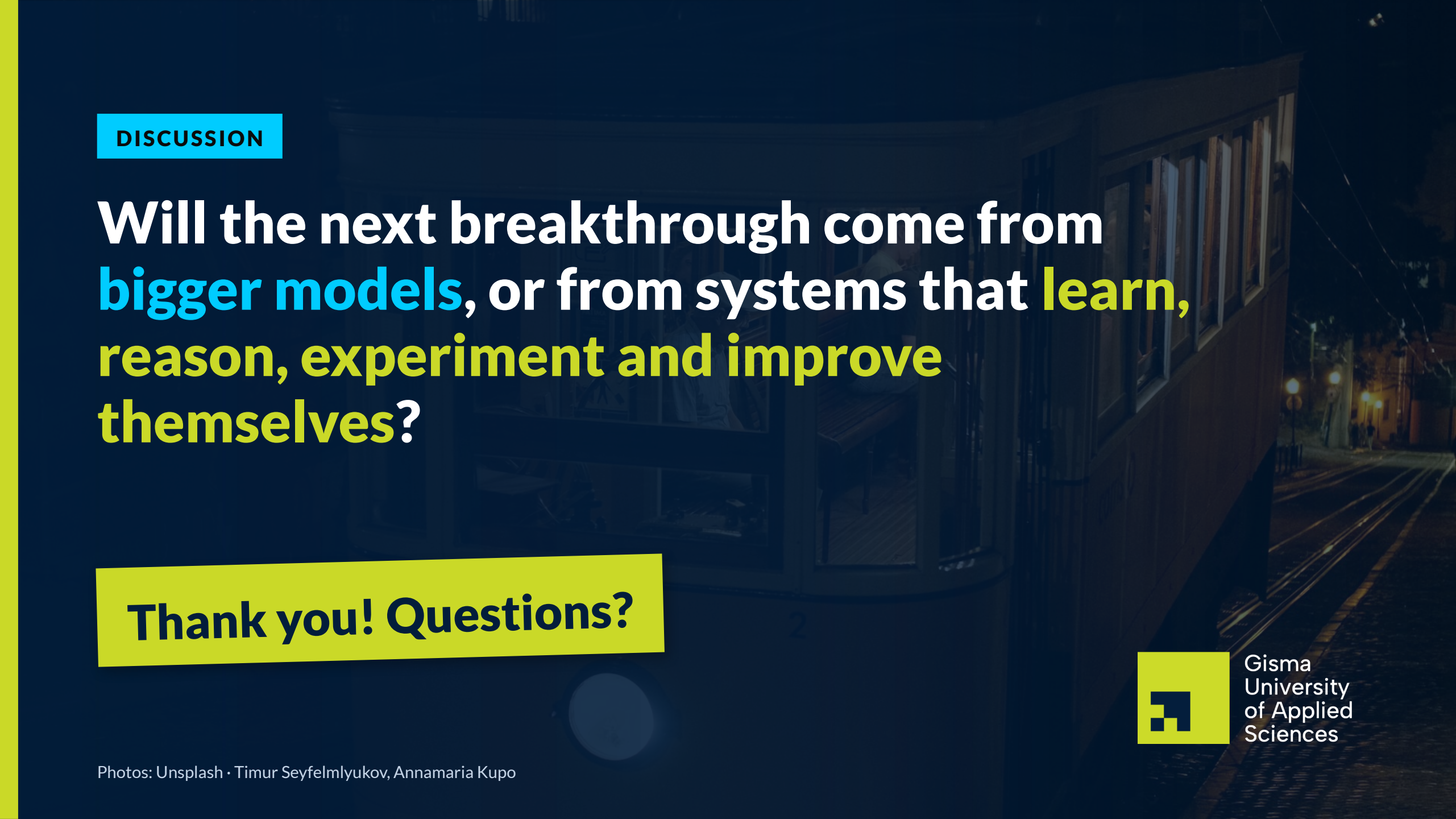
**Session 1: memory**

**Better judgement, not just bigger loops.**

DISCUSSION

# Will the next breakthrough come from bigger models, or from systems that learn, reason, experiment and improve themselves?

**Thank you! Questions?**



Gisma University of Applied Sciences

Photos: Unsplash · Timur Seyfelmlyukov, Annamaria Kupo